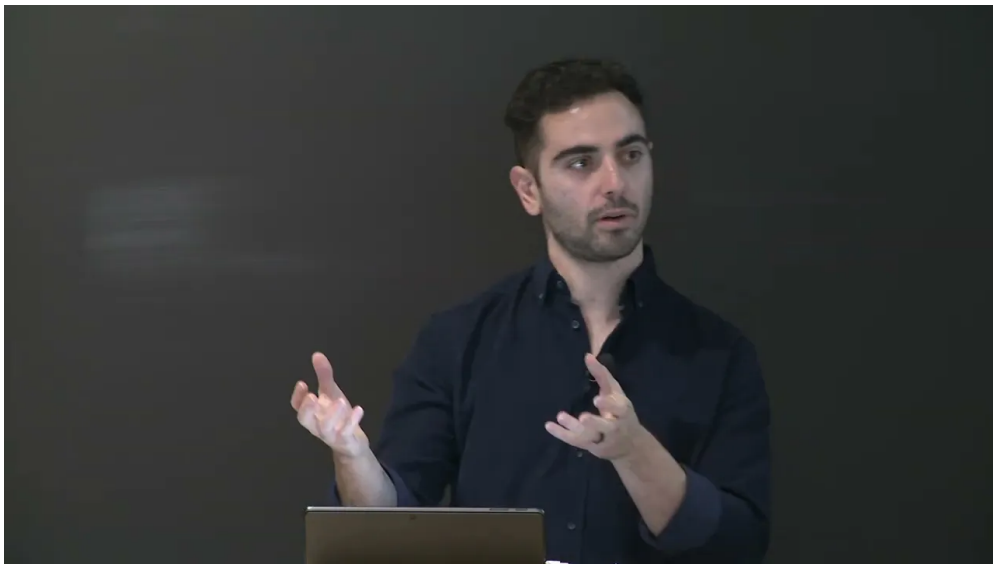


课程笔记

MIT RES.9-009 第 11 讲: Synaptic Plasticity and Reinforcement Learning

五道口纳什 & Codex

2026 年 4 月 15 日



视频作者/频道: MIT Video Productions (讲者: Haris Organzidis)

发布日期: 课程日期 2025-01-16; YouTube 上传 2025-04-10

视频时长: 10:52

视频链接: <https://www.youtube.com/watch?v=pBvgcIHK6GY>

目录

证据说明

本讲正文优先依据 MIT OCW 官方课程页 `learning.html`、YouTube 官方字幕与课程 `notebook` 的代码块摘要。课程页中存在的商业许可提示、以及无法获得的官方 `slides PDF`，不作为教学正文展开；相关遗漏记录在 `omission_log.jsonl` 中。

1 突触可塑性：从生物学直觉到学习规则

这讲的核心不是先讲“强化学习算法”，而是先把学习放回到突触层面：神经元之间的连接不是静态常数，而是会随着活动历史改变的参数。课程从最基础的生理直觉切入：**potentiation** 让连接更容易传递 spike，**depression** 则让连接变弱。这个变化可以发生在毫秒级，也可以跨越更长时间尺度，因此它既能解释快速适应，也能解释更慢的经验累积。

1.1 Hebbian 规则的最小形式

课程给出的经典入口是 Hebb 的命题：*neurons that fire together wire together*。如果两个通过同一 synapse 相连的神经元在时间上高度相关，那么这个 synapse 倾向于被强化。作为教材化写法，可以把最小的 Hebbian 更新概括为

$$\Delta w_{ij}(t) \propto s_i(t) s_j(t), \quad (1)$$

其中 s_i 与 s_j 分别表示 pre-synaptic 和 post-synaptic activity。这个式子不是课程里逐字写出的唯一公式，但它准确对应了课程对“同放电、同强化”的结构化解释。

关键理解

Hebbian learning 不是 reward learning。它只要求活动相关性，不要求外部奖励信号。因此它更像“局部统计规律”，而不是“任务目标导向”的优化器。

1.2 为什么突触可塑性会进入计算神经科学

课程强调，在建模时我们不只是想得到一个能跑的 ODE 系统，而是想要一个能够被数据检验的 dynamical system。也就是说，模型中的参数必须允许我们从观测信号反推出系统结构，或至少区分竞争性假设。突触可塑性之所以重要，是因为它把“连接强弱随时间变化”这件事直接变成了可计算、可验证的动力学。

教材化重写的边界

下面的更新式和解释使用了课程语义与标准计算神经科学记号做统一表达，便于自学；它们不是视频里逐行出现的唯一公式写法。

1.3 本章小结

这一部分建立了两件事：第一，synaptic plasticity 是学习的生物学载体；第二，Hebbian 规则提供了最基本的“活动相关性 $\rightarrow$ 权重更新”机制。后续章节所有 reinforcement learning 和 reward-modulated learning 的理解，都依赖这个局部更新直觉。

2 类别学习：从 cortico-cortical LTP 看到任务结构

课程把抽象的可塑性规则放入一个具体行为任务：category learning。动物看到一个 dot pattern，需要判断它属于 left 还是 right 类别。这个实验的关键并不只是“分类正确率”，而是它迫使模型在 trial-by-trial 的反馈中逐步形成稳定的 decision boundary。

2.1 任务语义与模型结构

官方讲述把这个任务翻译成了一个具身的 circuit：视觉输入进入 visual cortex，随后驱动 cortex-to-cortex 与 cortex-to-striatum 的连接，再由 action selection 形成左/右选择，最后通过 reward / no reward 反馈回到学习规则里。换句话说，模型不是纯抽象分类器，而是一个带有神经回路语义的 agent-environment system。

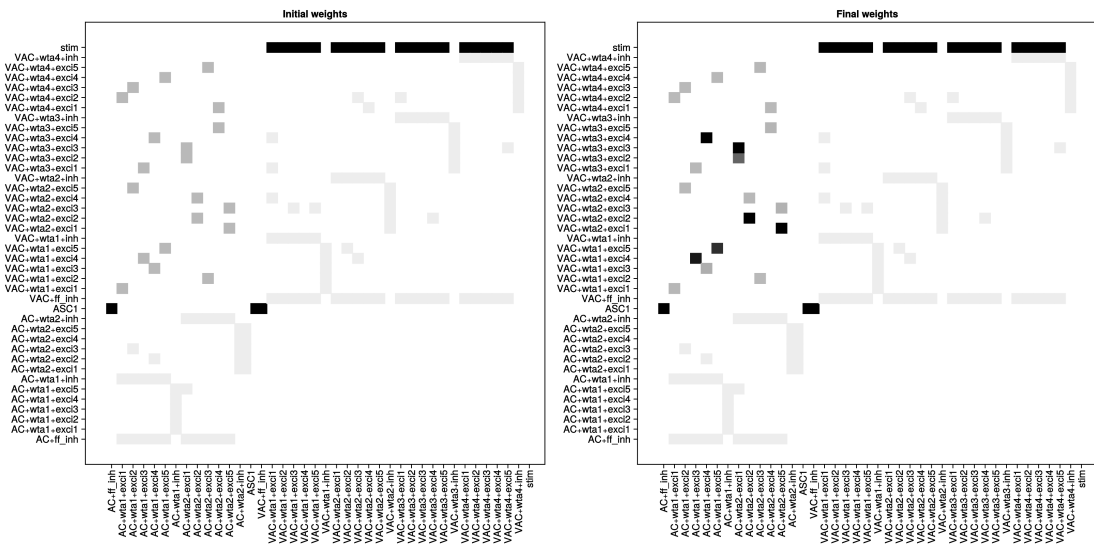


图 1：皮层-皮层学习前后的连接矩阵。左图为 initial weights，右图为 final weights。图源：MIT Neuroblox 官方课程页。

2.2 cortico-cortical LTP

课程指出，cortico-cortical synapses 中的 LTP 属于典型 Hebbian learning：pre-synaptic 和 post-synaptic activity 越相关，权重越倾向上升。更重要的是，这里并不只是“相关就增强”，而是有一个与 dopamine 相关的反转项，使得奖励结果可以在试次级别上改变权重走向。教材化地写成

$$\Delta w_{ij}^{cc}(t) = \eta s_i(t) s_j(t) (1 + \gamma \delta_t), \quad \delta_t = r_t - \hat{r}_t, \quad (2)$$

会更便于记忆。这里的 δ_t 就是 reward prediction error。注意，这个表达是标准化的教学重写；课程里对它的口头表述是“dopamine 相关的 reversal term”和“actual reward minus expected reward”。

奖励误差的角色

在课程语境中，dopamine 不是单纯的“奖励多一点/少一点”的口号，而是把行为反馈转成 synaptic update direction 的调制信号。正奖励可以维持或增强 potentiation, reward omission 或 punishment 则可能把权重推向相反方向。

2.3 本章小结

这一部分的重点是：课程把 Hebbian plasticity 放进具体的 category-learning 任务中，说明“局部相关性”可以通过 reward gating 变成“任务导向的学习”。图上的 initial/final weights 不是装饰，而是在显示回路如何从稀疏、未定向的连接走向能支撑行为的结构。

3 cortico-striatal 回路：奖励调制的强化学习

如果说前一节还停留在 cortex-to-cortex 的局部规则，那么这里就进入完整的 reinforcement learning 语义：视觉输入到 cortex，随后到 striatum，再到 action，再回到 reward feedback。课程把这个过程与 macaque 的 category learning 文献直接对齐，因此它并不是“任意设计的 toy problem”，而是一个生物学动机很强的 behavioral task。

3.1 强化学习的回路映射

在这个模型里，Agent 由图结构构成；Environment 则是 ClassificationEnvironment，里面包含多个刺激、每个刺激的真实类别，以及每个 trial 的结果。策略部分使用 greedy policy：比较 STR1 与 STR2 的活动，选择更高者所对应的行动。这个设计把抽象 RL 语义映射成了 Neuroblox 的具体对象。

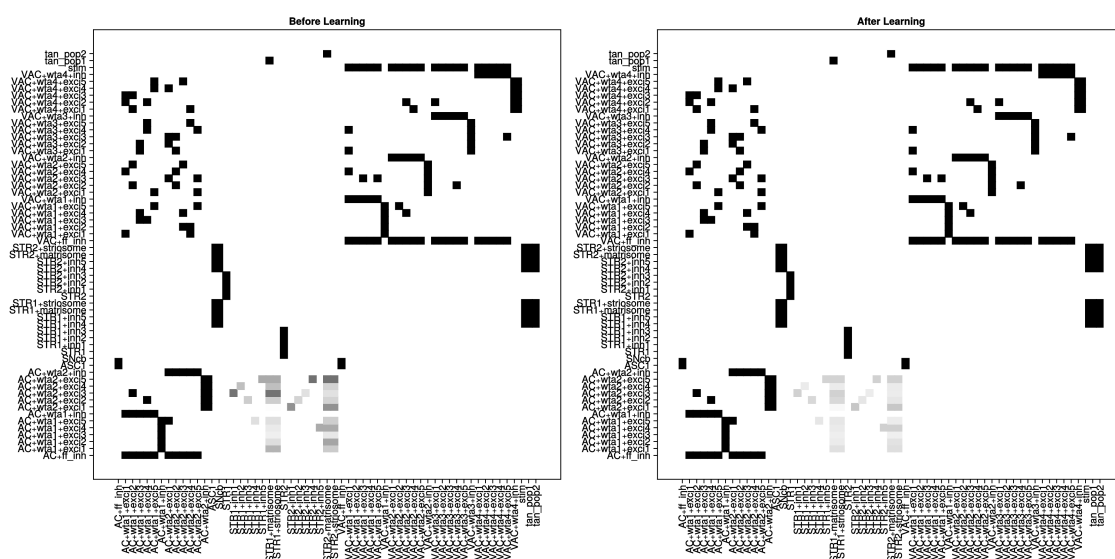


图 2: 完整 corticostriatal RL 模型的初始与学习后连接矩阵。左图为 Before Learning，右图为 After Learning。图源：MIT Neuroblox 官方课程页。

3.2 奖励误差与 dopamine

课程里最重要的桥梁，是把 reward prediction error 与 dopamine 调制联系起来。讲者明确说明， ϵ 表示 actual reward 与 expected reward 的差，并把它与 Q-learning、temporal difference learning 的误差项并列起来看。用标准 RL 记号重写，可以写成

$$\epsilon_t = r_t - \mathbb{E}[r_t], \quad \Delta w_{ij}(t) \propto \epsilon_t s_i(t) s_j(t). \quad (3)$$

这正是“synaptic plasticity $\rightarrow$ reinforcement learning”这条线的数学化表达。需要强调的是，课程并没有把生物回路简化成一个纯算法黑箱；相反，它一直在保留“哪个区域在调制、哪个连接在更新”的回路语义。

3.3 课程里的生物学 caveat

讲者在口头表述中对 dopamine 来源核团使用了 VTA 和 substantia nigra 的说法；从教材角度，不必在这里过度死磕核团命名，而应抓住更稳健的结论：一个全局奖励信号在调制局部 *Hebbian* 更新。这就是课程想表达的生物-算法桥梁。

3.4 本章小结

这一部分说明了：强化学习并不只是状态-动作-奖励的符号系统，它可以被具体地编码成回路内的可塑性机制。课程中最值得记住的是 reward prediction error 这一点，因为它同时连接了神经生物学、传统 RL 和 Neuroblox 的实现接口。

4 Notebook 结构、代码逻辑与参数敏感性

课程要求学员打开 `learning.ipynb`。从官方页面与代码块摘要看，这个 notebook 的骨架非常清晰：先搭模型，再定义环境与 agent，再跑多 trial 学习，最后看 adjacency matrix 和 trial-level trace。

4.1 核心代码流程

```
1 using Neuroblox
2 using OrdinaryDiffEq
3 using Random
4 using CairoMakie
5
6 time_block_dur = 90.0
7 N_trials = 100
8 trial_dur = 1000
9
10 image_set = CSV.read(Downloads.download(...), DataFrame)
11 agent = Agent(g; name=model_name, t_block=time_block_dur)
12 env = ClassificationEnvironment(stim, N_trials; name=:env, namespace=model_name)
13
14 trace = simulate(agent, env)
```

这段代码的逻辑不是“导入包并调用函数”这么简单。它实际上分成四层：第一层是数值求解与绘图；第二层是把刺激组织成环境；第三层是把 circuit graph 封装成 agent；第四层是让每个 trial 结束后自动更新权重。课程中强调，weight updates are automatically handled after each simulation when doing reinforcement learning in Neuroblox。

4.2 trace 的读法

课程示例里会查看 `trace.trial`、`trace.correct` 和 `trace.action`。这三个字段分别告诉你 trial 编号、是否答对、以及实际做出的动作。对于教材学习者来说，这比只看最终 accuracy 更有价值，因为它能暴露 learning curve、策略切换点和参数敏感性。

4.3 参数敏感性与练习

Challenge problem 直接点出了这讲的工程教训：这个简化版 cortico-striatal model 对参数非常敏感。比如 TAN 的 maximum activity $\kappa_{\max}$ 和 HebbianModulationPlasticity 的 learning rate 常数 K ，稍微偏离合适区间，系统就可能学不起来。这说明模型既有解释力，也有脆弱性。

自学时最该盯住的现象

不要只看最终“学没学会”，还要看 trial 过程中的权重漂移、正确率曲线和参数窗口。一个能解释学习的模型，必须能说明为什么在某些超参数下会失败。

4.4 本章小结

Notebook 的意义不只是演示一个能运行的例子，而是把“回路、环境、奖励、更新”四件事绑成一个可重复的实验流程。对于自学者，最值得复现的是初始权重、学习后权重和 trial-level 记录三者之间的对应关系。

5 总结与延伸

这一讲把三层东西串了起来：Hebbian plasticity 提供局部相关性更新，reward prediction error 把局部更新变成任务导向的 learning rule，而 cortico-striatal 回路则把这种学习落到可运行的 behavioral model。课程的高明之处在于，它没有把 reinforcement learning 讲成脱离生物学的算法，而是把它还原成一个可以在 Neuroblox 里逐步调试的 circuit-level process。

从研究视角看，这一讲的延伸方向有两个。第一，继续比较不同 reward-modulated plasticity 规则与标准 RL 算法之间的对应关系，特别是 Q-learning 和 temporal difference learning 的误差项如何被神经调制解释。第二，研究参数敏感性与可识别性：当模型对 $\kappa_{\max}$ 、 K 或输入分布非常敏感时，究竟是模型太简化，还是实验设计本身给出的信息不足。

延伸阅读

课程页参考文献中的两篇文献最值得回看：Pathak et al. (2024, bioRxiv) 提供了 corticostriatal micro-assemblies 的建模背景；Antzoulatos & Miller (2014, *Neuron*) 则把 prefrontal cortex 与 striatum 在 category learning 中的功能连接变化讲得非常清楚。